Database Management Systems (DBMS)

Study Notes for AI Personalized Learning & Study Planner RAG Testing

1. Introduction to DBMS

A Database Management System (DBMS) is software used to create, store, organize, retrieve, and manage data in a database. It provides an interface between users or applications and the stored data.

Main functions of a DBMS:

- Data storage and retrieval
- Data organization and efficient querying
- Data security and access control
- Transaction management
- Backup and recovery
- Concurrency control
- Maintaining data consistency and integrity

2. DBMS vs File System

Feature	File System	DBMS
Data redundancy	Usually higher	Can be controlled
Data consistency	Harder to maintain	Integrity constraints help
Security	Basic or application-dependent	Built-in access control
Concurrent access	Limited	Concurrency control supported
Backup and recovery	Often manual	Systematic mechanisms available
Querying	Requires application logic	SQL and query processing

3. Relational Database Model

The relational model represents data using relations, commonly implemented as tables. A table contains rows and columns. A row represents a record, while a column represents an attribute.

- **Relation:** A table containing related records.
- **Tuple:** A row in a relation.
- **Attribute:** A column describing a property of the data.
- **Domain:** The set of valid values for an attribute.

4. Keys

- **Primary Key:** Uniquely identifies each row. It cannot contain NULL values.
- **Candidate Key:** A minimal set of attributes that can uniquely identify a row.

- **Super Key:** A set of attributes that uniquely identifies a row, possibly with extra attributes.
- **Foreign Key:** An attribute that references a key in another table and helps maintain referential integrity.
- **Composite Key:** A key consisting of more than one attribute.

5. SQL Basics

SQL (Structured Query Language) is used to define, manipulate, query, and control data in relational databases.

- **DDL:** CREATE, ALTER, DROP, TRUNCATE
- **DML:** INSERT, UPDATE, DELETE
- **DQL:** SELECT
- **DCL:** GRANT, REVOKE
- **TCL:** COMMIT, ROLLBACK, SAVEPOINT

6. SQL Joins

A JOIN combines rows from two or more tables using a related column or join condition.

- **INNER JOIN:** Returns rows with matching values in both tables.
- **LEFT JOIN:** Returns all rows from the left table and matching rows from the right table.
- **RIGHT JOIN:** Returns all rows from the right table and matching rows from the left table.
- **FULL OUTER JOIN:** Returns matching rows and unmatched rows from both sides where supported.
- **CROSS JOIN:** Produces the Cartesian product of two tables.

7. Normalization

Normalization is a database design technique used to organize data into tables in order to reduce unnecessary redundancy and improve data integrity.

First Normal Form (1NF)

A relation is in 1NF when attributes contain atomic values and repeating groups are eliminated. Each field should contain a single value rather than a list of values.

Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and every non-key attribute is fully functionally dependent on the entire candidate key. This is particularly important when a table has a composite key.

Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and non-key attributes do not have inappropriate transitive dependencies on a key. The goal is to avoid storing facts that depend on another non-key attribute.

8. Functional Dependency

A functional dependency describes a relationship between attributes. If attribute X determines attribute Y, it can be written as $X \rightarrow Y$. This means that for a given value of X, the corresponding value of Y is uniquely

determined.

Example: In a student table, Student_ID → Student_Name means a particular Student_ID determines the student's name.

9. Transactions and ACID Properties

A transaction is a logical unit of database operations. A transaction should preserve database correctness even when failures or concurrent operations occur.

- **Atomicity:** A transaction is treated as an all-or-nothing operation.
- **Consistency:** A transaction should preserve defined database constraints.
- **Isolation:** Concurrent transactions should not improperly interfere with each other.
- **Durability:** Committed changes should survive failures.

10. Indexing

An index is a data structure that can speed up data retrieval for suitable queries. Indexes can improve read performance but also require storage and can add overhead to insert, update, and delete operations.

11. Common SQL Examples

Select all students

```
SELECT * FROM students;
```

Filter rows

```
SELECT * FROM students WHERE score >= 75;
```

Sort results

```
SELECT * FROM students ORDER BY score DESC;
```

Group data

```
SELECT subject, AVG(score) FROM performance GROUP BY subject;
```

Join tables

```
SELECT s.name, p.score FROM students s JOIN performance p ON s.student_id = p.student_id;
```

12. Quick Revision Questions

1. What is a DBMS and why is it used?
2. What is the difference between a primary key and a foreign key?
3. What is normalization?
4. What conditions should a relation satisfy for 1NF?
5. What is the difference between 2NF and 3NF?
6. What is a functional dependency?
7. What is an INNER JOIN?
8. What is the difference between WHERE and GROUP BY?

9. What are the four ACID properties?

10. Why can indexes improve query performance?